AI Study Companion

Technical & Project Report

Verified against the codebase in repository “PROJECT 2”.
Prepared: May 2026.
Product branding in the UI: “UNITEN AI” / UNITEN-focused assistant prompts (`backend/lib/prompts.ts`).

1. Executive summary

This document describes **AI Study Companion**, a full-stack web application comprising a React (Vite) frontend and a Next.js API backend backed by Supabase PostgreSQL, file storage, and external AI APIs. The scope of this report is limited to artefacts that exist in the repository: application code under `frontend/` and `backend/`, SQL migrations under `backend/supabase/migrations/`, deployment configuration `render.yaml`, and documentation under `docs/`. Root `README.md` summarizes run instructions and additionally refers to folders as `pixel-perfect-backend` / `pixel-perfect-frontend`; the on-disk sibling directories in this checkout are named `backend` and `frontend`.

2. Introduction and objectives

The system supports student workflows commonly needed for coursework: authenticated access, uploading study documents, generating summaries and document-grounded chat, quizzes (including AI-assisted generation flows exposed in the UI), retrieval-augmented “Ask AI” over indexed text, a writing-improvement tool, task scheduling, user profile endpoints, optional email flows, and administrative-style screens for analytics, quizzes, RAG uploads, and user status. Objectives evidenced in code include clear separation between SPA frontend and JSON API backend, cookie-based sessions, validation with Zod, and integration with LLM and embedding providers configured via environment variables.

3. Stakeholders and context

- **Primary users:** Students (default role `student` in schema) using study features.
- **Operators:** Whoever configures Supabase, Render (or equivalent host), SMTP, and API keys per `docs/HOW_TO_RUN.md` and `backend/.env.example`.
- **Institutional framing:** README and UI copy target UNITEN; LLM system prompts reference UNITEN students (`backend/lib/prompts.ts`).

4. System architecture

The frontend is a single-page application built with Vite and React; it calls the backend over HTTP with `credentials: "include"` for cookie-based authentication (`frontend/src/lib/api.ts` pattern, verified in Vitest mocks). The backend is Next.js 15 with API routes only under `backend/app/api/`; `backend/middleware.ts` sets CORS headers and handles `OPTIONS`. Persistence uses Supabase: PostgreSQL (with `pgvector` enabled in migration `001_initial_schema.sql`) plus Storage buckets referenced in HOW_TO_RUN (`documents`, `rag`). AI responsibilities are split: text generation via GROQ or Google Gemini (`backend/lib/llm.ts`, `LLM_PROVIDER`), embeddings via Jina AI (default) or HuggingFace Inference API (`backend/lib/embeddings.ts`, `EMBEDDING_PROVIDER`).

5. Repository layout (actual paths)

Path	Purpose
`frontend/`	Vite + React SPA, port 5173 in local docs.
`backend/`	Next.js API server, dev port 3001 per `package.json` script.
`docs/`	Project documentation (`HOW_TO_RUN.md`, `TESTING.md`, `MANUAL_TEST_CASES.md`, `FEATURES_AND_MODULES.md`, `INTEGRATION_CHECKLIST.md`, `PROJECT_STRUCTURE.md`, this report).
`render.yaml`	Render deployment blueprint for backend (Node) + frontend (static).

6. Functional requirements (verified in codebase)

Each item below maps to routes, pages, or libraries present in this repository.

6.1 Authentication and account lifecycle

- **FR-A1 Registration:** `POST /api/auth/register` — Zod-validated email and password (minimum 8 characters), bcrypt hash (cost factor 10), duplicate email handling, optional welcome email with timeout semantics (`backend/app/api/auth/register/route.ts`).
- **FR-A2 Credential login:** NextAuth Credentials provider; rejects unknown users, wrong passwords, and users with `is_active === false` (`backend/auth.ts`).
- **FR-A3 JWT session:** Session strategy JWT; `maxAge` 30 days; user `id` and `role` on token/session callbacks (`backend/auth.ts`).
- **FR-A4 Password reset:** `/api/auth/forgot-password`, `/api/auth/reset-password` with `password_reset_tokens` table (migration `004_password_reset_tokens.sql`).

- **FR-A5 Frontend auth shell:** `AuthContext`, public routes for login/register/forgot/reset, `ProtectedRoute` wrapping authenticated area (`frontend/src/App.tsx`, `frontend/src/contexts/AuthContext.tsx`).

The repository's `docs/MANUAL_TEST_CASES.md` includes a scenario expecting rejection of non-UNITEN registration email. The current `register` route validates format and password length only and does not enforce an `@uniten.edu.my` domain in `backend/app/api/auth/register/route.ts`. Automated API client tests assert handling of a backend error message mentioning UNITEN when the API returns that error (`frontend/src/lib/api.test.ts`).

6.2 Notes and document AI

- **FR-N1** List notes and upload notes: `GET /api/notes`, `POST /api/notes/upload` — PDF/DOCX/TXT, max 20 MB (`backend/app/api/notes/upload/route.ts`).
- **FR-N2** Note detail and delete: `GET/DELETE /api/notes/[id]`.
- **FR-N3** Summarization: `POST /api/summarize`.
- **FR-N4** Document chat: `POST /api/notes/chat`; UI on note detail (`frontend/src/pages/NoteDetail.tsx`).
- **FR-N5** Text extraction: `pdf-parse`, `mammoth`, plain text (`backend/lib/extract-text.ts`); Next config lists `pdf-parse` as external server package (`backend/next.config.ts`).

6.3 Quizzes

- **FR-Q1** List quizzes: `GET /api/quiz` — without `admin=true`, only `is_active` quizzes; with `admin=true`, all quizzes (same file as create).
- **FR-Q2** CRUD/metadata: `POST`/`PUT`/`DELETE` on `/api/quiz`, `/api/quiz/[id]`.
- **FR-Q3** Questions: add via `POST /api/quiz/[id]/questions`, delete with query (`/api/quiz/[id]/questions`).
- **FR-Q4** AI generation: `POST /api/quiz/generate`.
- **FR-Q5** Attempt lifecycle: submit `POST /api/quiz/[id]/submit`, review `GET /api/quiz/attempts/[attemptId]`.
- **FR-Q6 Smart quiz UI:** route `/quizzes/smart` (`frontend/src/App.tsx`, page `SmartQuiz.tsx`) — adaptive study flow consuming APIs as implemented in that page.

6.4 Ask AI (RAG)

- **FR-R1** Question answering: `POST /api/rag/ask` — embeds query, ensures user has at least one `rag_documents` row with `admin_id` equal to current user UUID, retrieves chunks via RPC `match_rag_chunks_for_user` (migration `005_match_rag_chunks_for_user.sql`) with fallback to `match_rag_chunks` then

filtering by document IDs; optional `courseFilter` substring match (`backend/app/api/rag/ask/route.ts`).

- **FR-R2** Admin RAG ingestion: `POST /api/admin/rag/upload`, list `GET /api/admin/rag/documents` — chunking via `chunkText` (`backend/lib/chunk.ts`), max upload 50 MB (`admin/rag/upload/route.ts`).

6.5 Writing coach

- **FR-W1** `POST /api/writing/improve` — improves student text (`backend/app/api/writing/improve/route.ts`), UI `WritingCoach.tsx`.

6.6 Schedule / tasks

- **FR-T1** `GET/POST/PUT/DELETE /api/tasks` (`backend/app/api/tasks/route.ts`); frontend `Schedule.tsx`, notification hook `useTaskNotifications.ts`.

6.7 User profile

- **FR-P1** `GET/PUT /api/user/profile`.

6.8 Admin-facing APIs (path naming)

- **FR-M1 Analytics:** `GET /api/admin/analytics`.
- **FR-M2 Users:** `GET /api/admin/users`, `PATCH /api/admin/users` (supports `is_active` toggling as used by frontend tests and admin UI patterns).
- **FR-M3 Email tooling:** `POST /api/admin/email`, `POST /api/admin/email/test`; `Nodemailer` + Gmail-compatible SMTP in `backend/lib/email.ts`.

6.9 Access control (as implemented today)

Observation: `requireAdmin` in `backend/lib/auth.ts` is documented in-file as intentionally a no-op after a product decision; `AdminRoute` in the frontend is a pass-through. Endpoints still call `requireAdmin(session)` for compatibility, but that call does not restrict by role. Effective protection for most student data relies on querying by authenticated `user_id` from the session and on NextAuth guarding routes with `requireAuth()`.

7. Non-functional requirements (evidence in repo)

Type	Requirement	Evidence
Security — passwords	Hashes with bcrypt	`bcryptjs` in register / authorize (`backend/app/api/auth/register/route.ts`, `backend/auth.ts`)
Security — transport/session	JWT session; HTTPS-only cookie settings when backend URL uses HTTPS (`useSecureCookies` branch in `backend/auth.ts`)	
Security — input validation	Zod schemas on API bodies	Representative routes: register, rag ask, tasks, etc.
Security — CORS	Middleware sets allow-origin per localhost or configured `FRONTEND_URL`	`backend/middleware.ts`
Maintainability	TypeScript frontend and backend	`tsconfig.json` files, `typescript` devDependencies
Reliability — LLM misuse	Guardrails in `callLlm` (Vitest-covered)	`backend/lib/llm.ts`, tests in `backend/lib/llm.test.ts`
Testability	Unit/integration-style tests via Vitest (no external services for core suites)	`vitest.config.ts` in frontend and backend; see §10
Deployability	Declared Render services + explicit build paths	`render.yaml`

8. Specifications — technology stack

8.1 Frontend (`frontend/package.json`)

Concern	Technology (representative)
Runtime/UI	React 18, TypeScript, Vite 5
Routing	react-router-dom 6
Server/cache state	TanStack React Query 5
Forms / validation	react-hook-form, zod, hookform/resolvers
Styling/components	Tailwind CSS 3.x, Radix UI primitives (many `@radix-ui/*` packages), class-variance-authority, lucide-react, shadcn-style component folder under `src/components/ui/`
Charts misc.	recharts, embla-carousel, cmdk, vaul drawer, sonner toasts
Quality tooling	ESLint 9, Vitest 3, Testing Library React, jsdom

8.2 Backend (`backend/package.json`)

Concern	Technology
Framework	Next.js ~15
Auth	next-auth 5 beta, Credentials provider
Database client	@supabase/supabase-js 2.x
LLM APIs	groq-sdk, @google/generative-ai
Documents	pdf-parse, mammoth
Email	nodemailer 7
Validation	zod 3
Quality tooling	ESLint 9, eslint-config-next, Vitest 2

8.3 Environment configuration (verified files)

`backend/.env.example` lists: Supabase URL and service role key, `DATABASE_URL`, `NEXTAUTH_SECRET`, `NEXTAUTH_URL`, `FRONTEND_URL`, `LLM_PROVIDER` (`groq|gemini`), `GROQ_API_KEY`, optional Gemini keys, `EMBEDDING_PROVIDER` (`jina` default | `huggingface`), `JINA_API_KEY`, optional HuggingFace keys, optional SMTP. Frontend expects `VITE_API_URL` per `docs/HOW_TO_RUN.md`.

`render.yaml` declares `NODE_VERSION` 20 for the backend Web Service and static publish of `frontend/dist`; visible secret keys include Supabase and NextAuth, Gemini and HuggingFace embedding model env; operators must align live env with embedding/LLM code paths (Groq/Jina documented for local HOW_TO_RUN; Render excerpt centres Gemini + HuggingFace — treat as deployment-specific).

9. Backend API catalogue (filesystem)

Every handler below exists under `backend/app/api/.../route.ts` (HTTP methods implemented per-file; representative groups):

- Admin cluster: analytics (`GET`), users (`GET/PATCH`), email (`POST`), email test (`POST`), RAG documents (`GET`), RAG upload (`POST`).
- Auth: `auth/[...nextauth]` (NextAuth), register, forgot-password, reset-password.
- notes, notes/upload, notes/[id], notes/chat.
- Quiz tree: quiz, quiz/generate, quiz/[id], quiz/[id]/questions, quiz/[id]/submit, quiz/attempts/[attemptId].
- rag/ask, summarize, tasks, user/profile, writing/improve.

Concrete file list: `admin/analytics/route.ts`, `admin/email/route.ts`, `admin/email/test/route.ts`, `admin/rag/documents/route.ts`, `admin/rag/upload/route.ts`, `admin/users/route.ts`, `auth/[...nextauth]/route.ts`, `auth/forgot-password/route.ts`, `auth/register/route.ts`, `auth/reset-password/route.ts`, `notes/chat/route.ts`, `notes/route.ts`, `notes/upload/route.ts`, `notes/[id]/route.ts`, `quiz/generate/route.ts`, `quiz/route.ts`, `quiz/[id]/route.ts`, `quiz/[id]/questions/route.ts`, `quiz/[id]/submit/route.ts`, `quiz/attempts/[attemptId]/route.ts`, `rag/ask/route.ts`, `summarize/route.ts`, `tasks/route.ts`, `user/profile/route.ts`, `writing/improve/route.ts`.

10. Data model and migrations

Initial schema `001_initial_schema.sql` creates tables: `users` (with `role` check), `documents`, `summaries`, `quizzes`, `questions`, `options`, `quiz_attempts`, `answers`, `tasks`, `writing_sessions`, `rag_documents`, `rag_chunks` with `vector(384)`, IVFFlat index, RLS enabled on tables, and function `match_rag_chunks`. Follow-on migrations: `002_fix_vector_dimension.sql` (vector/RPC alignment), `003_add_user_status.sql` (`is_active`), `004_password_reset_tokens.sql`, `005_match_rag_chunks_for_user.sql` (per-owner search joining `rag_documents`).

11. Frontend routing (actual)

Public: `/login`, `/register`, `/forgot-password`, `/reset-password`. Authenticated: `/`, `/notes`, `/notes/:id`, `/quizzes`, `/quizzes/smart`, `/quizzes/:id/play`, `/quizzes/:id/results`, `/ask-ai`, `/writing-coach`, `/schedule`, `/settings`, plus `/admin`, `/admin/documents`, `/admin/quiz`, `/admin/users`, fallback `*` (`frontend/src/App.tsx`).

12. Testing — what has been executed and documented

12.1 Manual testing

`docs/MANUAL_TEST_CASES.md` specifies step-by-step scenarios for Authentication, Notes, Summarization, Quiz (student + admin flows), Writing Coach, Tasks, RAG/Ask AI and document chat, Admin pages, and Dashboard integration. `docs/TESTING.md` points readers to that file and to `docs/INTEGRATION_CHECKLIST.md` for coverage expectations. These documents exist in `docs/`; executing them requires a running local stack per `HOW_TO_RUN.md`.

12.2 Automated tests (Vitest) — runs performed for this report

Commands: `cd frontend && npm run test` and `cd backend && npm run test` on May 14 2026. Results:

Package	Files	Tests (passed)
Frontend	`src/test/example.test.ts` (placeholder), `src/lib/api.test.ts`, `src/lib/api-extra.test.ts`	32
Backend	`lib/chunk.test.ts`, `lib/extract-text.test.ts`, `lib/llm.test.ts`	21

Frontend suites (behaviour summarized from source): API client URLs, methods, JSON bodies/responses for session, registration, notes, tasks, quizzes (admin flag), quiz submit/update/delete, Ask RAG, writing improvement, analytics, users, email, RAG upload document list, attempt results, sign-out, and error propagation from non-OK HTTP responses.

Backend suites: `chunkText` splitting/overlap/whitespace behaviour; `isAllowedMime` and plain-text extraction / unsupported MIME errors; `callLlm` branches for Groq (missing key, successful completion, alleged unsafe completion replacement, empty message) and Gemini (missing key, success, blocked content).

12.3 Other checks

`npm run lint` scripts exist on both workspaces but were not rerun as a gate for this document. `backend/test-email.mjs` exists as a standalone mail smoke script (optional operator use).

13. Documentation set delivered with the project

- `README.md` — overview, doc index, run/test summary.
- `docs/HOW_TO_RUN.md` — prerequisites, Supabase setup, env templates, troubleshooting.
- `docs/TESTING.md` — manual vs automated testing entry points (note: it still names `pixel-perfect-frontend`/`pixel-perfect-backend` while this tree uses `frontend`/`backend`).
- `docs/MANUAL_TEST_CASES.md`, `docs/INTEGRATION_CHECKLIST.md`, `docs/FEATURES_AND_MODULES.md`, `docs/PROJECT_STRUCTURE.md`.
- Backend developer notes: `backend/README.md`.

14. Traceability and limitations of this report

Assertions in this PDF were cross-checked against source files and migrations on the date above. Older narrative documents (for example `docs/FEATURES_AND_MODULES.md`) may describe role-gated admins or HuggingFace-only embeddings; the running code should be taken as authoritative where it diverges — notable differences already called out in §6.1 and §6.9. Deployment URLs in descriptive docs (e.g. sample Render hostnames) are illustrative unless they match an active environment configuration outside this repository.

End of report.